

Parcours de graphes	2
Introduction.....	2
Problème	3
Définition	3
Définition	3
Différents types de parcours	4
Parcours & Arborescences.....	5
Structure intermédiaire : le tableau visite.....	5
Parcours en profondeur d'abord	6
Principe.....	6
Exemple.....	6
Algorithme : Parcours en Profondeur.....	10
// Version récursive.....	12
Version itérative	13
Exemple.....	14
.....	14
.....	14
Parcours en Largeur	15
Principe.....	15
Algorithme du parcours en largeur.....	17
Exemple.....	20

PARCOURS DE GRAPHS

Introduction

Les algorithmes de parcours de graphes servent de base à bon nombre d'algorithmes.

Ces algorithmes nécessitent un examen exhaustif des sommets et des arcs (resp. Arêtes) du graphe.

Le problème consiste à déterminer un ordre sur les sommets traités par exemple.

L'ordre des sommets visités induit une numérotation des sommets.

Un parcours de graphe est un outil pour étudier une propriété globale du graphe :

Le graphe est-il connexe ?

Le graphe est-il fortement connexe ?

Le graphe comporte-t-il un circuit ?

Le graphe est-il biparti ?

Etc

Problème

Parcourir un graphe à partir d'un sommet donné, c'est explorer (traiter, visiter) l'ensemble des sommets et une seule fois.

Le problème consiste à déterminer un ordre sur les visites des sommets.

Pour faciliter la tâche, on choisira l'ordre lexicographique devant un branchement (plusieurs possibilités)

Définition

Le sommet de départ, fixé à l'avance, dont on souhaite explorer tous les descendants est appelé **racine** de l'exploration.

Définition

Le parcours d'un graphe à partir d'un sommet de départ nommé Racine est une suite ordonnée de sommets telle que :

Le sommet racine est le premier sommet de la suite

Chaque sommet apparaît une fois et une seule fois

Tout sommet sauf la racine est adjacent à un sommet placé avant lui dans cette suite

Différents types de parcours

On distingue deux types de parcours :

Parcours en profondeur : consiste à partir d'un sommet de départ, à suivre un chemin le plus loin possible (jusqu'à un cycle), puis à faire des retours en arrière pour reprendre tous les chemins ignorés précédemment.

Parcours en largeur : consiste à explorer les sommets du graphe niveau par niveau à partir d'un sommet de départ choisi.

Parcours & Arborescences

Les parcours en profondeur et/ou en largeur donnent naissance à des arbres associés (forêt d'arborescences), dont les sommets sont ceux du graphe et dont les arcs sont définis par l'ordre de parcours.

La structure de chacun des arbres dépend, en plus du type de parcours choisi, du sommet sélectionné comme sommet de départ.

Le sommet de départ est considéré comme racine de l'exploration.

Si le graphe est connexe, on qualifie ces arbres de couvrants, car ils couvrent tous les sommets du graphe.

Structure intermédiaire : le tableau visite

On utilise une structure intermédiaire qui nous permettra de marquer les sommets déjà explorés, afin de détecter qu'il ne faut plus les traiter.

Ceci peut se faire à l'aide d'un tableau de booléens indicés par les numéros de sommets et initialisés à faux.

Parcours en profondeur d'abord

Principe

Le parcours consiste à visiter tous les sommets d'un graphe donné.

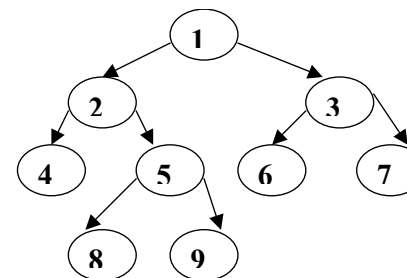
A partir d'un sommet donné, on procède de la manière suivante :

On suit un chemin le plus longtemps possible en allant visiter à chaque fois le premier sommet non encore exploré, voisin du sommet où l'on se trouve.

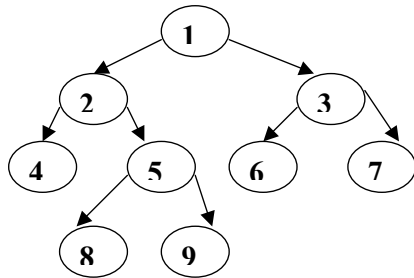
Lorsque ceci n'est plus possible, on revient sur ses pas pour explorer les possibilités laissées de côté.

On parcourt en premier lieu les chemins les plus récemment écartés.

Exemple



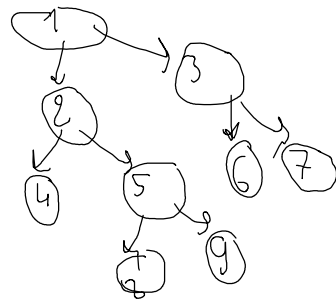
Ordre du parcours en profondeur à partir du sommet 1 est :
[1 2 4 5 8 9 3 6 7]



Sommet Départ = 1

Succ(1)={2, 3}

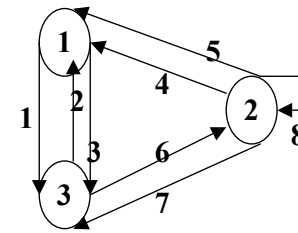
PP(1)



Visite=

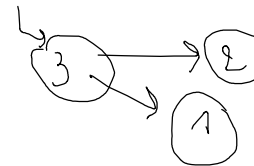
V	V	V	V	V	V	V	V	V
---	---	---	---	---	---	---	---	---

Ordre de traitement = {1, 2, 4, 5, 8, 9, 3, 6, 7}



Sommet Depart=3

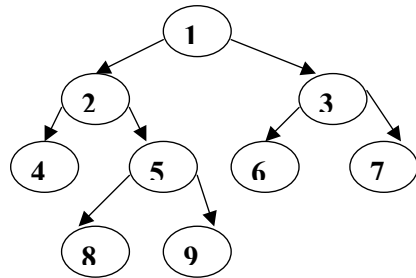
PP(3)



Visite

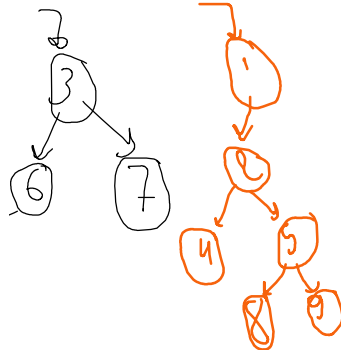
V	V	V
---	---	---

Ordre de traitement={3, 1, 2}



Sommet Départ = 3
PP(3, G)

Sommet Départ = 1
PP(1, G)



D'après l'ordre lexicographique, on choisira le sommet 1 comme sommet de départ.

Visite=

V	V	V	V	V	V	V	V	V
---	---	---	---	---	---	---	---	---

Ordre de traitement = {3, 6, 7, 1, 2, 4, 5, 8, 9}

Algorithme : Parcours en Profondeur

Principe :

Lorsqu'on rencontre un sommet non encore exploré, on traite récursivement en profondeur le sous graphe que l'on peut atteindre à partir de ce sommet.

Hypothèse : On suppose que les sommets sont numérotés de 1 à n.

Parcours en profondeur d'un graphe à partir d'un sommet de départ

Procédure parcoursProfondeur(sd, g)

P.F sd : Entier (E) // sommet de départ

g : Graphe (E)

Début

//Initialisation

$n \leftarrow \text{nbSom}(g)$

visite $\leftarrow \text{allocTab}(n)$

pour chaque sommet s variant de 1 à n faire

visite[s] $\leftarrow$ faux

finPour

s $\leftarrow$ sd

nbSomVisite $\leftarrow$ 0

finParcours $\leftarrow$ faux

//Traitement

Tantque non finParcours faire

// appel de la version récursive**reParcoursProfondeur(s, visite, g, nbSomVisite)****// ou appel de la version itérative****// itParcoursProfondeur(s, visite, g, nbSomVisite)**

Si nbsomVisite < n

s ← somSuivant(s, n, visite)

Si non

finParcours ← Vrai

FSi

FinTq

visite ← libTab(visite)**FIN****// Version 2 : Traitement****Répéter****// appel de la version récursive****reParcoursProfondeur(s, visite, g, n, nbSomVisite)****// ou appel de la version itérative****// itParcoursProfondeur(s, visite, g, n, nbSomVisite)**

Si nbsomVisite < n

s ← somSuivant(s, n, visite)

FinSi

Tantque nbsomVisite < n**// Version récursive****Procédure reParcoursProfondeur(s, visite, g, n, nbSomVisite)**

P.F s : Entier (E)

visite : Tableau (E/S)

g : graphe (E)

n : Entier (E)

nbSomVisite : Entier (E/S)

Début

Si n < nbSomVisite alors

Visite[s] ← vrai**traiter(s)****nbSomVisite ← nbSomVisite+1**

Pour t ∈ Γ(s) faire

Si non visite[t] alors

reParcoursProfondeur(t, visite, g, n, nbSomVisite)

Fsi

FPour

FinSi

Fin

Version itérative

Procédure itParcoursProfondeur(s, visite, g, n, nbSomVisite)

P.F s : Entier (E)
 visite : Tableau (E/S)
 g : graphe (E)
 n : Entier (E)
 nbSomVisite : Entier (E/S)

Debut

p ← initPile()

p ← empiler(s, P)

arret ← faux

Tantque non estVidePile(P) et non arret faire

u ← sommetPile(P)

p ← depiler(P)

Si non visite[u] alors

traiter (u)

nbSomVisite ← nbSomVisite+1

Si n = nbSomVisite alors

arret ← vrai

Sinon

visite[u] ← vrai

Pour t variant du Dernier au Premier Successeur de u faire

si non visite[t] alors

p ← empiler(t, P)

FinSi

Fpour

FinSi

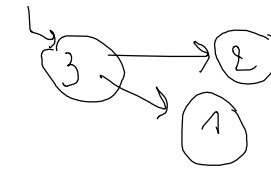
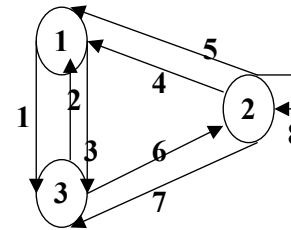
FinSi

FinTq

[....]

FIN

Exemple



Sommet Depart=3

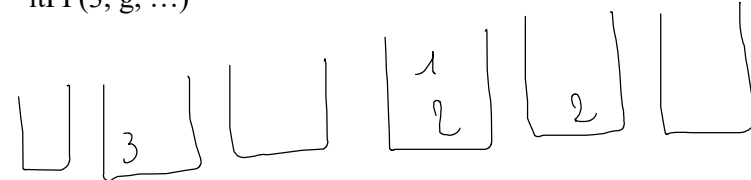
Visite

X	X	V
---	---	---

nbSomVisite ← 0 }

Ordre de traitement={3, 1, 2}

itPP(3, g, ...)



Parcours en Largeur

Soit un graphe $G=(S, A)$ et un sommet origine s .

Le parcours en largeur emprunte systématiquement les arcs (resp arêtes) de G pour découvrir tous les sommets accessibles depuis s .

Principe

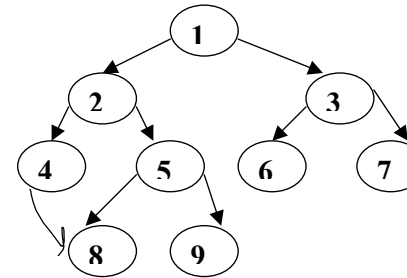
On commence au sommet s et on le marque comme étant sommet visité.

L'ensemble des sommets adjacents à s sont visités.

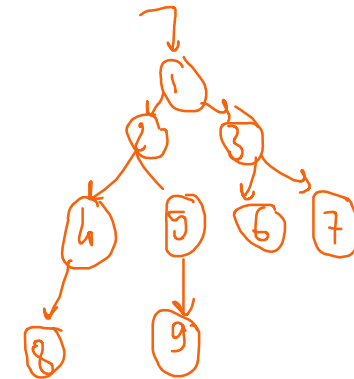
Ensuite, les sommets non traités de ses sommets sont à leurs tour visités et ainsi de suite jusqu'à la visite de l'ensemble des sommets du graphe.

Il construit une arborescence de parcours en largeur de racine s , qui contient tous les sommets accessibles depuis s .

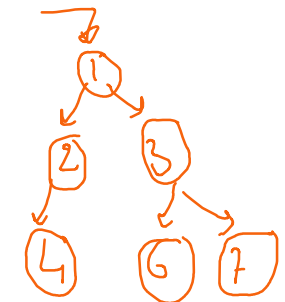
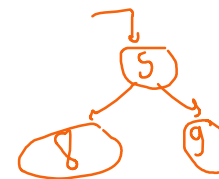
Exemple



Sommet Départ = 1



Sommet Départ = 5



Algorithme du parcours en largeur

Hypothèse : On suppose que les sommets sont numérotés de 1 à n.

Procédure `parcoursLargeur(sd, g)`

P.F `sd` : Entier (E) // sommet de départ
 `g` : Graphe (E)

Début

//Initialisation

`n` ← `nbSom(g)`

`visite` ← `allocTab(n)`

pour chaque sommet `s` variant de 1 à `n` faire

`visite[s]` ← faux

finPour

`s` ← `sd`

`nbSomVisite` ← 0

`finParcours` ← faux

//Traitement

tantque non `finParcours` faire

`itParcoursLargeur(s, visite, g, n, nbSomVisite)`

 Si `nbSomVisite` < `n` alors

`s` ← `somSuivant(s, n, visite)`

 Si non

`finParcours` ← Vrai

 FSi

finTq

`visite` ← `libTab(visite)`

FIN

//Version 2 : Traitement

Répéter

`itParcoursLargeur(s, visite, g, n, nbSomVisite)`

Si `nbSomVisite` < `n` alors

`s` ← `somSuivant(s, n, visite)`

FinSi

Tantque `nbSomVisite` < `n`

Procédure itParcoursLargeur(s, visite, g, n, nbSomVisite)

P.F s : Entier (E)
visite : Tableau (E/S)
g : graphe (E)
n : Entier (E)
nbSomVisite : Entier (E/S)

Debut

F ← initFile()

F ← enfiler(s, F)

arret ← faux

tantque non estVideFile(F) et non arret faire

u ← sommetFile(P)

f ← defiler(F)

Si non Visite[u] alors

Visite[u] ← vrai

traiter (u)

nbSomVisite ← nbSomVisite+1

Si n= nbSomVisite alors

arret ← vrai

Sinon

Pour t variant du Premier au Dernier $\Gamma(u)$ faire

Si non visite[t] alors

F ← enfiler(t, F)

Finsi

Fpour

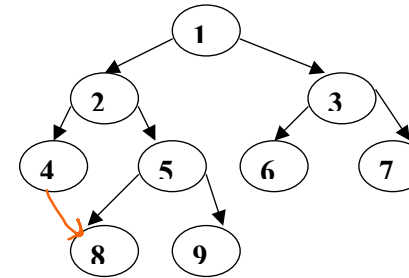
FinSi

Finsi

Ftq

FIN

Exemple



SD=5

nbSomViste ← 9

visite

V	V	V	V	V	V	V	V	V
1	2	3	4	5	6	7	8	9

Ordre de traitement = {5, 8, 9, 1, 2, 3, 4, 6, 7}

